

The Monte Carlo Method

Antonio M.N. Lima

DEE/CEEI/UFCG

Outline

- 1 Introduction & Historical Context
- 2 Mathematical Formulation
- 3 Relevant Extensions
- 4 MATLAB Example 1 — Estimating π
- 5 MATLAB Example 2 — Normal Distribution
- 6 Summary

What is the Monte Carlo Method?

Definition

The **Monte Carlo method** is a broad class of computational algorithms that rely on *repeated random sampling* to obtain numerical results.

- Named after the famous casino in Monaco.
- Formalised during the **Manhattan Project** by Stanisław Ulam and John von Neumann (late 1940s).
- Core idea: generate many joint outcomes of random processes and observe the behaviour of response values characterised by probability density functions.

Key Insight

Replace an *analytical* computation with a *statistical* one — trade mathematical difficulty for computational effort.

The Fundamental Problem

Suppose we wish to evaluate:

$$\mu = \mathbb{E}[g(X)] = \int g(x) dF(x),$$

where X is a random variable with distribution F and g is a measurable function.

Difficulty: The expected value μ exists but we may have *no closed-form formula* to compute it.

Special Case — Definite Integral

$$I = \int_a^b f(x) dx = (b - a) \mathbb{E}[f(U)], \quad U \sim \text{Uniform}(a, b).$$

The Monte Carlo Estimator

Generate N i.i.d. samples $X_1, X_2, \dots, X_N \sim F$.

MC Approximation

$$\hat{\mu}_N = \frac{1}{N} \sum_{i=1}^N g(X_i)$$

This replaces the true distribution F with the **empirical distribution** $\hat{F}_N$ (equal weights $1/N$).

For definite integrals:

$$\hat{I}_N = \frac{b-a}{N} \sum_{i=1}^N f(U_i), \quad U_i \sim \text{Uniform}(a, b).$$

Theoretical Justification

Two pillars of Monte Carlo:

① **Strong Law of Large Numbers (SLLN):**

$$\hat{\mu}_N \xrightarrow{a.s.} \mu \quad \text{as } N \rightarrow \infty.$$

② **Central Limit Theorem (CLT):** If $\sigma^2 = \text{Var}[g(X)] < \infty$, then

$$\sqrt{N}(\hat{\mu}_N - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

Convergence Rate

$\mathcal{O}(N^{-1/2})$ — **independent of the dimensionality** of the problem.

Error bound: $\varepsilon \approx \frac{\sigma}{\sqrt{N}} z_{\alpha/2}.$

Markov Chain Monte Carlo (MCMC)

When direct sampling from F is **intractable**, construct an ergodic Markov chain $\{X_t\}$ whose stationary distribution is F .

Metropolis–Hastings Algorithm

- 1 Given current state X_t , propose $Y \sim q(\cdot | X_t)$.
- 2 Accept with probability

$$\alpha = \min\left(1, \frac{f(Y) q(X_t | Y)}{f(X_t) q(Y | X_t)}\right).$$

- 3 If accepted set $X_{t+1} = Y$; otherwise $X_{t+1} = X_t$.

- **Gibbs Sampling:** special case where each component is sampled from its full conditional.

Importance Sampling

Sample from an alternative distribution $h(x)$ to **reduce variance**:

$$\mu = \int g(x) \frac{f(x)}{h(x)} h(x) dx = \mathbb{E}_h \left[g(X) \frac{f(X)}{h(X)} \right].$$

- The ratio $w(x) = f(x)/h(x)$ is the **importance weight**.
- Optimal choice of h can *dramatically* reduce variance.

Estimator

$$\hat{\mu}_N^{\text{IS}} = \frac{1}{N} \sum_{i=1}^N g(X_i) \frac{f(X_i)}{h(X_i)}, \quad X_i \sim h.$$

Further Extensions

Mean-Field Particle Methods Systems of N interacting particles; each sample interacts with the empirical measure. As $N \rightarrow \infty$, the empirical measures converge to the deterministic distribution of a nonlinear Markov chain.

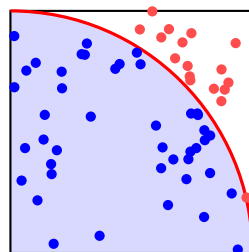
Quasi-Monte Carlo (QMC) Replace pseudo-random sequences with **low-discrepancy sequences** (Sobol, Halton). Convergence rate approaches $\mathcal{O}(N^{-1})$ under smoothness conditions (Koksma–Hlawka inequality).

Multilevel Monte Carlo (MLMC) Introduced by M. Giles; exploits a hierarchy of approximations at different resolutions for variance reduction at reduced cost — particularly effective for SDEs and PDEs.

Estimating π : The Idea

- Inscribe a quarter-circle of radius 1 inside the unit square $[0, 1]^2$.
- Area of quarter-circle = $\pi/4$.
- Fraction of random points inside $\rightarrow \pi/4$ as $N \rightarrow \infty$.
- Therefore:

$$\hat{\pi} = 4 \frac{\#\{\text{inside}\}}{N}.$$



$[0, 1]^2$

MATLAB Code: Estimating π (Core)

```
1 %% Monte Carlo Estimation of Pi
2 rng(42); % reproducibility
3 N_values = [1e2, 1e3, 1e4, 1e5, 1e6];
4
5 fprintf('%-15s %-15s %-15s\n', 'N', 'Pi Estimate', 'Abs Error');
6 fprintf('%s\n', repmat('-', 1, 45));
7
8 for k = 1:length(N_values)
9     N = N_values(k);
10    X = rand(N, 1); % uniform in [0,1]
11    Y = rand(N, 1);
12    inside = (X.^2 + Y.^2) <= 1; % inside quarter circle?
13    pi_est = 4 * sum(inside) / N;
14    abs_err = abs(pi_est - pi);
15    fprintf('%-15d %-15.6f %-15.6e\n', N, pi_est, abs_err);
16 end
```

MATLAB Code: Visualisation & Convergence

```
1 %% Scatter plot for N = 10 000
2 N = 10000; X = rand(N,1); Y = rand(N,1);
3 inside = (X.^2 + Y.^2) <= 1;
4 figure('Color','w');
5 scatter(X(inside),Y(inside),2,'b','filled'); hold on;
6 scatter(X(~inside),Y(~inside),2,'r','filled');
7 theta = linspace(0,pi/2,200);
8 plot(cos(theta),sin(theta),'k-','LineWidth',2);
9 axis equal; xlim([0 1]); ylim([0 1]);
10 title(sprintf('MC Pi (N=%d, est=%.4f)',N,4*sum(inside)/N));
11 legend('Inside','Outside','Quarter Circle','Location','sw');
12
13 %% Convergence study
14 rng(42); N_max = 1e5;
15 X = rand(N_max,1); Y = rand(N_max,1);
16 cum_inside = cumsum((X.^2+Y.^2)<=1);
17 n = (1:N_max)'; pi_run = 4*cum_inside./n;
18 figure('Color','w');
19 semilogx(n, pi_run,'b-'); hold on; yline(pi,'r--','LineWidth',2);
20 fill([n;flipud(n)],[pi+2./sqrt(n);flipud(pi-2./sqrt(n))],...
21      'r','FaceAlpha',0.1,'EdgeColor','none');
22 xlabel('N'); ylabel('\pi estimate'); grid on;
```

Approximating the Standard Normal Distribution

Goal A — PDF: Generate N samples via the *Box–Muller* transform and build a histogram.

$$Z = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad U_1, U_2 \sim \text{Uniform}(0, 1).$$

Goal B — CDF via MC integration:

$$\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^z e^{-t^2/2} dt = \frac{1}{2} + \frac{1}{\sqrt{2\pi}} \int_0^z e^{-t^2/2} dt \quad (z \geq 0).$$

For $z < 0$: use symmetry $\Phi(z) = 1 - \Phi(-z)$.

MC estimator for the integral $\int_0^z e^{-t^2/2} dt$:

$$\hat{I}_N = \frac{z}{N} \sum_{i=1}^N e^{-U_i^2/2}, \quad U_i \sim \text{Uniform}(0, z).$$

MATLAB Code: PDF Approximation (Box-Muller)

```
1 %% Part A: PDF approximation via Box-Muller sampling
2 rng(42); N = 1e6;
3 U1 = rand(N,1); U2 = rand(N,1);
4 Z = sqrt(-2*log(U1)) .* cos(2*pi*U2); % Box-Muller
5
6 figure('Color','w');
7 histogram(Z, 200, 'Normalization','pdf',...
8 'FaceColor',[0.3 0.6 0.9],'EdgeAlpha',0.3);
9 hold on;
10 z_grid = linspace(-5,5,1000);
11 phi_true = (1/sqrt(2*pi)) * exp(-z_grid.^2/2);
12 plot(z_grid, phi_true, 'r-', 'LineWidth',2);
13 title(sprintf('MC Approx of N(0,1) PDF (N = %d)', N));
14 xlabel('z'); ylabel('f(z)');
15 legend('MC Histogram','True PDF','Location','ne');
16 grid on;
```

MATLAB Code: CDF via MC Integration

```
1 %% Part B: CDF approximation
2 N_mc = 1e6; z_eval = linspace(-4,4,200);
3 Phi_mc = zeros(size(z_eval));
4
5 for k = 1:length(z_eval)
6     z = z_eval(k);
7     if abs(z) < 1e-12
8         Phi_mc(k) = 0.5;
9     elseif z > 0
10        U = z * rand(N_mc,1);
11        Phi_mc(k) = 0.5 + (1/sqrt(2*pi)) * z * mean(exp(-U.^2/2));
12    else
13        zp = -z; U = zp * rand(N_mc,1);
14        Phi_mc(k) = 1-(0.5 + (1/sqrt(2*pi))*zp*mean(exp(-U.^2/2)));
15    end
16 end
17
18 Phi_true = 0.5*(1 + erf(z_eval/sqrt(2)));
19 figure('Color','w');
20 subplot(2,1,1);
21 plot(z_eval,Phi_mc,'b.','MarkerSize',4); hold on;
22 plot(z_eval,Phi_true,'r-','LineWidth',2);
23 title('MC Approx of Standard Normal CDF');
24 legend('MC','True','Location','se'); grid on;
25 subplot(2,1,2);
26 plot(z_eval, abs(Phi_mc-Phi_true),'k-');
27 title('Absolute Error'); xlabel('z'); grid on;
```

MATLAB Code: Convergence for $\Phi(1.96)$

```
1 %% Part C: Convergence for Phi(1.96)
2 z_target = 1.96;
3 Phi_exact = 0.5*(1 + erf(z_target/sqrt(2)));
4 N_range = round(logspace(2,6,50));
5 Phi_est = zeros(size(N_range));
6 conf = zeros(size(N_range));
7
8 for k = 1:length(N_range)
9     Nk = N_range(k);
10    U = z_target * rand(Nk,1);
11    fv = exp(-U.^2/2);
12    Phi_est(k) = 0.5 + (1/sqrt(2*pi))*z_target*mean(fv);
13    sig = z_target*std(fv)/sqrt(2*pi);
14    conf(k) = 1.96*sig/sqrt(Nk);
15 end
16
17 figure('Color','w');
18 errorbar(N_range, Phi_est, conf, 'b-o','MarkerSize',3);
19 hold on; yline(Phi_exact,'r--','LineWidth',2);
20 set(gca,'XScale','log');
21 title(sprintf('Convergence: \\Phi(%.2f)',z_target));
22 xlabel('N'); ylabel('\\Phi(z)');
23 legend('MC +/- 95% CI',...
24        sprintf('True = %.4f',Phi_exact),'Location','se');
25 grid on;
```


Summary Table

Aspect	Detail
Convergence Rate	$\mathcal{O}(N^{-1/2})$, dimension-independent
Error Bound	$\varepsilon \approx \sigma z_{\alpha/2} / \sqrt{N}$ (CLT)
Strength	Scales to high dimensions; trivially parallelisable
Weakness	Slow for low-dim. smooth problems; variance can be large
Key Extensions	MCMC, Importance Sampling, QMC, MLMC, Particle Filters

Concluding Remarks

- Monte Carlo remains one of the most **versatile and powerful** tools in computational stochastic processes.
- Its **dimension-independent** convergence rate makes it indispensable in:
 - Statistical mechanics
 - Financial engineering
 - Bayesian inference
 - Uncertainty quantification
- With nothing more than a *uniform random number generator*, one can:
 - Approximate transcendental constants (π).
 - Compute distribution functions lacking closed-form antiderivatives ($\Phi(z)$).

Does it make sense?